```
import asyncio
import time
from datetime import datetime
from apscheduler.schedulers.asyncio import AsyncIOScheduler

from config import CHECK_INTERVAL_SECONDS, SIGNAL_COOLDOWN_MINUTES, SYMBOL_NAME
from indicators import full_analysis
from signals import generate_signal
from news import get_gold_news
from bot import send_signal, send_text

# Track last sent signal to enforce cooldown
_last: dict = {"signal": None, "ts": 0}


async def check_and_signal():
    now_str = datetime.now().strftime("%H:%M:%S")
    print(f"[{now_str}] Scanning {SYMBOL_NAME}...")

    try:
        analysis = full_analysis()
        news     = get_gold_news()
        sig      = generate_signal(analysis, news)

        direction  = sig.get("signal")
        confidence = sig.get("confidence", 0)
        print(f"  → {direction} | confidence: {confidence}%")

        if direction == "NO_TRADE":
            return

        # Cooldown: skip if same direction sent recently
        elapsed = (time.time() - _last["ts"]) / 60
        if _last["signal"] == direction and elapsed < SIGNAL_COOLDOWN_MINUTES:
            print(f"  ⏳ Cooldown ({elapsed:.1f}/{SIGNAL_COOLDOWN_MINUTES} min) -
            return

        await send_signal(sig)
        _last["signal"] = direction
        _last["ts"]     = time.time()

    except Exception as e:
        print(f"  ❌ Error: {e}")
```

```
async def main():
    print(f"🚀 {SYMBOL_NAME} Scalping Bot starting...")
    await send_text(
        f"🤖 *{SYMBOL_NAME} Scalping Bot is online*\n"
        f"_Scanning every {CHECK_INTERVAL_SECONDS}s — 1m / 5m / 15m analysis_"
    )

    scheduler = AsyncIOScheduler()
    scheduler.add_job(check_and_signal, "interval", seconds=CHECK_INTERVAL_SECOND
    scheduler.start()

    await check_and_signal()   # run immediately on launch

    try:
        while True:
            await asyncio.sleep(1)
    except (KeyboardInterrupt, SystemExit):
        print("Bot stopped.")


if __name__ == "__main__":
    asyncio.run(main())
```